1. Asemanarile si deosebirile dintre efectele proiectiilor perspectiva si cele ale proiectiilor axonometrice.(3)

2. Prin ce se deosebesc proiectiile oblice de proiectiile axonometrice?(3)

3. Care dintre urmatoarele tipuri de proiectii nu este o transformare afina? Justificati.(3)

4. Care sunt parametrii care definesc volumul vizual al proiectiei perspectiva in OpenGL?(3) Care este planul in care se efectueaza proiectia?

5. Care sunt parametrii care definesc sistemul de coordonate observator? Care sunt valorile implicite ale acestor parametrii in OpenGL?(4)

6. Determinarea fetelor auto obturate ale poliedrelor convexe. Conditia de vizibilitate atunci cand determinarea este efectuata: a) in coordonate globale; b) in coordonate de decupare.(5)

7. Algoritmul Z-buffer. Descriere. Cand se executa? Depth complexity?(6)

8. Care dintre urmatoarele afirmatii sunt corecte?(8)

9. Algoritmul BSP(8)

10. Reflexia difuza a luminii intr-un punct al unei suprafete 3D. (10)

11. Aproximarea reflexiei difuze.(11)

12. Aproximarea reflexiei speculare a luminii intr-un punct al unei suprafete 3D, luminata de o sursa punctiforma (modelul empiric). (11)

13. Calculul culorilor fragmentelor la afisarea suprafetelor 3D folosind modelul Gouraud**.** Calculul incremental al culorilor. Avantajele si dezavantajele modelului Gouraud in comparatie cu alte modele. (Phong)(12)

14. Comparatie intre modelul Gourraud si modelul Phong(13)

15. Calculul culorilor fragmentelor la afisarea suprafetelor 3D folosind modelul Phong. Avantajele si dezavantajele modelului Phong in comparatie cu alte modele.(13)

16. Introducerea umbrelor in imagini prin metoda volumelor de umbra (fara implementare cu OpenGL). (14)

17. Introducerea cetei in imagini (15)

18. Ray Tracing **(17)**

19. Pentru decuparea vectorilor 2D: Descriere, Pasii algoritmului pentru decuparea vectorului din figura (vezi sus figura cu un patrat taiat de o linie)(18)

20. Decuparea vectorilor in algoritmul Cohen-Sutherland (fara implementare)(19)

21. Algoritmul Bresenham pentru rasterizarea vectorilor: alegerea punctelor spatiului discret, implementare in C, generalizarea algoritmului pentru vectori de orice panta.(20)

22. Algoritmul Bresenham pentru rasterizarea cercurilor: alegerea punctelor spatiului discret, implementare in C, functia de afisare a punctelor de pe cerc. (22)

23. Transformarea de vizualizare in OpenGL. Parametrii care definesc sistemul de coordonate observatoare. Care este functia de OpenGL care permite definirea acestor parametrii?(24)

24. Forma generala (in pseudocod) a algoritmului de rasterizare a suprafetelor poligonale. Care e definitia punctului interior a unui poligon, pe care se bazeaza algoritmul? Cate puncte de intersectie trebuie sa se

obtina atunci cand o linie raster traverseaza un varf al poligonului?(25)

25. Modelul de reflexie Phong (modelul de iluminare locala) pentru calculul luminii reflectate intr-un punct al unei suprafete 3D. Precizati care sunt componentele luminii reflectate si care sunt calculate. (27)

26. Care este matricea prin care se transforma un varf din sistemul coordonator obiect in sistemul coordonator de decupare? Care este volumul de decupare? Cand este efectuata decuparea primitivelor in banda grafica?(28)

27. Care sunt parametrii prin care este definit sistemul de coordonate observator (atasat camerei virtuale)? Care sunt valorile implicite ale acestor parametrii? Care este produsul de matrici prin care se obtine matricea transformarii de vizualizare(din sistemul de coordonate globale in sistemul de coordonate observator). (28)

28. Care sunt caracteristicile proiectiilor axonometrice? Scrieti produsul de matrici prin care se poate obtine proiectia axonometrica a unui punct. (29)

29. Cum se aleg pixelii de pe traseul unui vector in algoritmul Bresenham pentru rasterizarea vectorilor? Cum se generalizeaza algoritmul pentru rasterizarea vectorilor de orice panta? (30)

30. Cum sunt efectuate urmatoarele operatii in algoritmul Ray-tracing (33)

31. Cum poate fi utilizat algoritmul BSP pentru eliminarea obiectelor nevizibile din banda grafica (Object culling)? (35)

1. Precizati pe scurt care sunt asemanarile si deosebirile dintre efectele proiectiilor perspectiva si cele ale proiectiilor axonometrice.

Asemanari:

a) Redau mai multe fete ale obiectului proiectat.

b) Pot reprezenta un plan 2D un spatiu 3D.

c) Unghiurile se conserva pentru fetele obiectului paralele cu planul de proiectie

Deosebiri:

a) La proiectiile perspectiva, proiectiile liniilor paralele care nu sunt paralele cu planul de proiectie converg catre un punct din plan, numit punct de convergenta; proiectiile axonometrice sunt afine - conserva paralelismul liniilor.

b) Obiectele aflate la distanta mare se micsoreaza la perspectiva; in cazul proiectiilor axonometrice nu.

2. Prin ce se deosebesc proiectiile oblice de proiectiile axonometrice?

Proiectii oblice: plan de proiectie perpendicular pe o axa principala

Proiectii axonometrice: planul de proiectie nu e perpendicular pe niciuna din axele sistemului de coord.

Proiectii oblice:proiectorii nu sunt perpendiculari pe planul de proiectie

Proiectii axonometrice: proiectorii sunt perpendiculari pe planul de proiectie

3. Care dintre urmatoarele tipuri de proiectii nu este o transformare afina? Justificati.

Proiectia perspectiva nu este afina, adica nu conserva paralelismul liniilor (celelalte tipuri de proiectii erau paralele). In cazul proiectiei perspectiva, liniile care nu sunt paralele cu planul de proiectie converg catre un punct numit punct de convergenta.

4. Care sunt parametrii care definesc volumul vizual al proiectiei perspectiva in OpenGL? Care este planul in care se efectueaza proiectia?

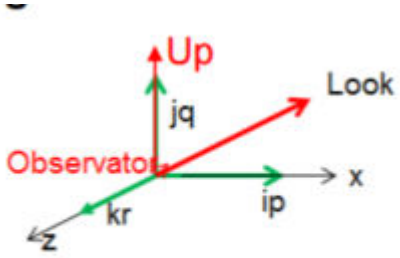
Volumul vizual al proiectiei perspectiva (View frustum): trunchi de piramida delimitat de planul din faţă (near) si planul din spate (far) al vederii, cu varful in pozitia observatorului.
Se efectueaza proiectia perspectiva în planul din faţă (in fereastra vizuala).

Proiectia perspectiva este reprezentata de un trunchi de piramida (frustum) definit prin cele 2 planuri, **cel din apropiere** si **cel din departare**, cat si de deschiderea unghiurilor de vizualizare pe cele 2 axe, OX si OY

Matricea de proiectie in acest caz poate fi calculata cu ajutorul functiei `glm::perspective` ce primeste ca si parametri deschiderea unghiului de vizualizare pe orizontala (**Field of View - FoV**), raportul dintre latimea si inaltimea ferestrei de vizualizare (**aspect ratio**), cat si distanta pana la cele 2 planuri zFar si zNear.

5. Care sunt parametrii care definesc sistemul de coordonate observator? Care sunt valorile implicite ale acestor parametrii in OpenGL?

1. poziţia observatorului (camerei): originea sistemului de coordonate globale
2. direcţia în care priveşte observatorul: direcţia negativa a axei OZ
3. direcţia sus a planului de vizualizare: direcţia pozitivã a axei OY



6. Determinarea fetelor auto obturate ale poliedrelor convexe. Conditia de vizibilitate atunci cand determinarea este efectuata: a) in coordonate globale; b) in coordonate de decupare.

Tip back face culling:

- Poate reduce substantial numarul de poligoane procesate in banda grafica.

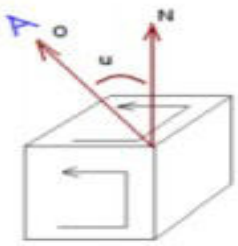
Algoritmul-Observatorul si obiectul sunt raportati la acelasi sistem de coordonate.

-Conturul fiecarei fețe a obiectului este orientat in sens trigonometric atunci cand poliedrul este vazut din exterior.

-Observatorul este situat in afara obiectului. Fața este vizibila daca : $0 <= u < 90^0$

N: normala la fața

O: vectorul orientat catre observator



Determinarea fetelor auto-obturate poate fi efectuata:

1. In sistemul coordonatelor globale (in care este definit si observatorul)

Produsul scalar:

$N \bullet O = |N| * |O| * \cos(u) \rightarrow \cos(u) = N \bullet O / (|N| * |O|) = Nu \bullet Ou$

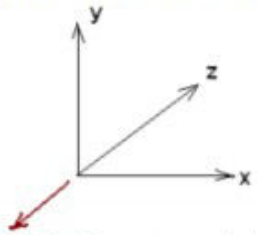
Conditia de vizibilitate: Nu • Ou > 0 (produsul scalar al versorilor)

2. In sistemul de coordonate observator

3. In sistemul coordonatelor de decupare (dupa aplicarea transformarii “model-view-projection”)

3. In sistemul coordonatelor de decupare (dupa aplicarea transformarii "model-view-projection")

In cazul 3:



Nu • Ou = [nx, ny, nz] • [0, 0, -1] = - nz

Conditia de vizibilitate devine: nz <0

O[0, 0, -1] - este acelasi pentru orice punct al unui obiect

Observatorul este la infinit, pe axa z negativa.

7. Algoritmul Z-buffer. Descriere. Cand se executa? Depth complexity?

- Algoritm de eliminare a partilor nevizibile ale primitivelor grafice.

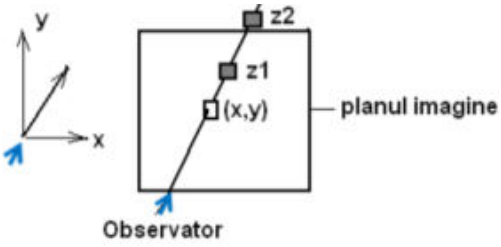
- Executat de GPU, fiind integrat in procesul de rasterizare a primitivelor (dupa transformarea în poarta de afişare a vârfurilor)

- Observatorul este la infinit, pe axa z negativa

- Asupra primitivelor se efectueaza proiectie ortografica(de ce asa?) in planul XOY

Doua fragmente aflate pe acelasi proiector, rezultate din rasterizarea a doua primitive diferite, au coordonata z diferita:

➢ Fragmentul avand coordonata z mai mica va fi afisat în pixelul (x,y).



Primitivele grafice (triunghiuri, linii) in care a fost descompusa scena 3D sunt rasterizate in ordinea in care au fost transmise din programul de aplicatie (nu sunt sortate). Buffer-ul imagine (frame buffer) este actualizat pe masura rasterizarii primitivelor, astfel: daca (fragmentul curent se proiecteaza in pixelul (x,y) si z-fragment < z-fragment afisat in pixelul (x,y))

=> actualizeaza culoarea pixelului (x,y) in bufer-ul imagine, la culoarea fragmentului curent

Rezulta: este necesar sa se memoreze coordonatele z ale fragmentelor afisate in pixelii

imaginii

Z-buffer: - tablou bidimensional cu numar de elemente egal cu numarul de pixeli ai suprafetei de afisare - în memoria placii grafice

Algoritm:

*Initializeaza buffer-ul imagine la culoarea de fond

*Initializeaza Z-buffer la coordonata z maxima (z=1)

Pentru fiecare fragment f(x,y,z) rezultat din rasterizarea unei primitive *

calculeaza culoarea fragmentului

//in urma rasterizarii nu rezulta doar pixelii?

//testul de vizibilitate (adancime)

daca z< Z-buffer[y][x] atunci

{ Z-buffer[y][x] = z

*actualizeaza culoarea pixelului (x,y) in buffer-ul imagine folosind culoarea fragmentului f

}

-Ultima transformare din lantul de transformari efctuate asupra varfurilor 3D conserva coordonata z a fiecarui varf.

Aprecieri asupra algoritmului z-Buffer

1) Numarul de comparatii de valori z pentru un pixel = numarul de fragmente care se proiecteaza in acel pixel. Timpul de calcul tinde sa devina independent de numarul de poligoane: in medie, numarul de pixeli acoperiti de un poligon este invers proportional cu numarul de poligoane.

2) “Depth complexity”: numarul de suprascrieri ale unui pixel in buffer-ul imagine, la generarea unui cadru imagine .

* calcul culoare fragment: model iluminare, utilizare texturi

3) Pentru o precizie buna, este necesara o reprezentare pe 32 biţi a valorilor din zbuffer.

8. Care dintre urmatoarele afirmatii sunt corecte?

a. Algoritmul z-buffer necesita rasterizarea poligoanelor scenei 3D in ordinea descrescatoare a distantei lor fata de observator - **Fals**

b. Bufferul z memoreaza coordonatele z ale fragmentelor de primitive care sunt vizibile intr-o imagine - **Adevarat**

c. Bufferul z se actualizeaza pentru fiecare fragment rezultat din rasterizarea primitivelor dintr-un cadru imagine - **Fals (testul de depth)**

d. Algoritmul z-buffer este exectutat de procesoarele placii grafice - **Adevarat**

9. Algoritmul BSP

Cum se contruieste arborele BSP al unei scene 3D si ce se memoreaza in nodurile sale?

Fiecare nod al arborelui corespunde unui plan de partitionare a spatiului 3D (de regula, planul unui poligon al scenei)

Fiecare plan de partitionare imparte spatiul in 2 semi-spatii:

–cel din faţa planului (de aceeasi parte cu normala la plan)

–cel din spatele planului

Se incepe cu un poligon oarecare din lista (de regula primul), pentru care se creeaza nodul radacina al arborelui

Primitivele din semi-spatiul "faţă" formeaza "lista-faţă", care va genera subarborele "faţă" al nodului. Primitivele din semi-spatiul "spate" formeaza "lista-spate", care va genera subarborele "spate" al nodului.

Se alege un poligon din lista-faţă si se creaza nodul radacina al subarborelui "faţă, etc.

Cand trebuie sa fie reconstruit arborele BSP al unei scene?

La modificarea pozitiei obiectelor din scena.

Eu cred ca arborele trebuie modificat (nu reconstruit) atunci cand scena contine obiecte dinamice. Construirea arborelui BSP este costisitoare, si de aceea exista adaptari ale alogoritmului BSP care sa evite reconstructia arborelui.

Ex: 1. Pentru fiecare cadru imagine se insereaza in arborele BSP cu obiecte statice noduri corespunzatoare obiectelor dinamice.

2. Se transmite arborele bsp nemodificat si obiectele dinamice, iar eliminarea partilor obturate de obiectele dinamice se face la rasterizare, folosind algoritmul z-buffer.

Cum se afiseaza o scena reprezentata printr-un arbore BSP?

- Tine cont de pozitia observatorului

- Afisare “back-to-front” : se incepe cu primitiva cea mai indepartata de observator

- Se porneste din radacina arborelui si se avanseaza in arbore pana la frunze, in functie de pozitia observatorului fata de planul atasat fiecarui nod.

- In fiecare nod: se avanseaza in subarborele nodului care este opus observatorului fata de plan.

```
void afisareBSPcull (arbore * A, Pozitie Observator)
{ daca (! A) return;
  daca (Observator este in semispatiul fata al planului radacinii) atunci
    daca (planul nodului radacina nu intersecteaza volumul vizual) atunci
            afisareBSPcull(A→fata)
    altfel { afisareBSPcull(A->spate); *afisare primitive din nodul radacina; afisareBSPcull(A->fata);}
  altfel
    daca (planul nodului radacina nu intersecteaza volumul vizual) atunci
            afisareBSPcull(A→spate)
    altfel { afisareBSPcull(A->fata); *afisare primitive din nodul radacina; afisareBSPcull(A->spate);}
}
```

7

Scrieti functia de afisare arborelui BSP.

```
void afisareBSP(arbore * A, Pozitie Observator)
{ daca (! A) return;
  daca (Observator este in semispatiul fata al planului radacinii) atunci
    { afisareBSP(A->spate); *afisare primitive din nodul radacina; afisareBSP(A->fata);}
  altfel
    { afisareBSP(A->fata); *afisare primitive din nodul radacina; afisareBSP(A->spate);}
}
```

Cum poate fi utilizat algoritmul BSP pentr eliminarea poligoanelor nevizibile din banda grafica?

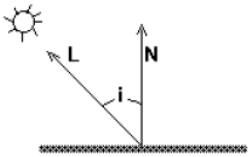
Daca planul de partitionare al unui nod nu intersecteaza volumul vizual, atunci numai subarborele aflat de aceeasi parte cu volumul vizual va fi afisat, celalalt subarbore fiind eliminat din banda grafica.

10. Reflexia difuza a luminii intr-un punct al unei suprafete 3D.

Aproximarea reflexiei difuze intr-un punct al unei suprafete 3D

- Lumina reflectată difuz de o suprafaţă este dispersată regulat în toate direcţiile.
- **Legea lui Lambert** defineşte reflexia luminii provenite de la o **sursă punctiformă**, de către un difuzor perfect:

$$Id = Isursa * kd * \cos(i) \qquad 0 <= i <= \pi/2$$



Reflexia difuza

Isursa – este intensitatea luminii incidente (provenita de la sursa de lumina)

Kd – este coeficientul de difuzie a luminii incidente, dependent de materialul suprafetei: suprafata reflecta/absoarbe/transmite lumina in anumite proportii: $0<=kd<=1$

Id este intensitatea luminii reflectate difuz de suprafata

- Dacă i este mai mare ca π/2, suprafaţa nu primeşte lumină de la sursă (sursa de lumină se află în spatele suprafeţei).

11. Aproximarea reflexiei difuze.

- Stiind că

$$\cos(i)=L\cdot N / (|L|\cdot|N|) = Lu\cdot Nu$$

rezulta ecuatia care modeleaza lumina reflectata difuz:

❖ $Id\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda * kd\lambda * (Lu \cdot Nu)$

$Cd = [IdR, IdG, IdB]$ – culoarea luminii reflectate difuz

Pentru a include si cazul in care $i > \pi/2$ (lumina de la sursa nu ajunge in punctul considerat):

❖ $I\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda * kd\lambda * \max((Lu \cdot Nu), 0)$

12. Aproximarea reflexiei speculare a luminii intr-un punct al unei suprafete 3D, luminata de o sursa punctiforma (modelul empiric).

Modelul Phong pentru aproximarea reflexiei speculare intr-un punct al unei suprafete 3D:

$$Is\lambda = Isursa\lambda * w(i,\lambda) * \cos(\alpha)^n$$

$w(i, \lambda)$ este funcţia de reflectanţă, i - unghiul de incidenţă iar λ - lungimea de undă a luminii incidente

In practica, $w(i, \lambda)$ este înlocuită cu o constantă determinată experimental, numită **coeficientul de reflexie speculară al materialului, notat** $ks\lambda$.

❖ Modelul practic al reflexiei speculare:

$$Is\lambda = Isursa\lambda * ks\lambda * \cos(\alpha)^n$$

$\cos(\alpha)=R\cdot V / (|R|\cdot|V|) = Ru\cdot Vu$

Rezulta:

$$Is\lambda = Isursa\lambda * fat * ks\lambda * (Ru \cdot Vu)^n$$

Pentru a include si cazul in care $\alpha = 90$:

$$Is\lambda = Isursa\lambda * fat * ks\lambda * \max((Ru \cdot Vu)^n, 0)$$

13. Calculul culorilor fragmentelor la afisarea suprafetelor 3D folosind modelul Gouraud. Calculul incremental al culorilor. Avantajele si dezavantajele modelului Gouraud in comparatie cu alte modele.(Phong)

Culorile fragmentelor interioare poligonului sunt obţinute la momentul rasterizarii, prin interpolarea liniară a culorilor din vârfuri.

1. Culoarea asociata unui varf se calculeaza folosind normala in varf:

- normala este asociata varfului in programul de aplicatie
- poate fi calculata exact din ecuatia analitica a suprafetei pe care se afla varful
- poate fi aproximata ca medie aritmetica a normalelor la feţele adiacente in varf

2. La rasterizarea unui poligon, GPU calculeaza culoarea fiecarui fragment rezultat astfel:

- prin interpolarea liniara a culorilor varfurilor, pentru fragmentele de pe laturi
- prin interpolare liniara intre culorile capetelor fiecarui segment interior, pentru fragmentele interioare poligonului.

Culoarea fragmentului este intrare pentru programul fragment shader.

Aprecieri asupra modelului Gouraud

- Se integreaza foarte usor in algoritmii de rasterizare
- Calcule simple pentru culoarea unui fragment (calcul incremental) => rapid
- Nu permite calculul luminii reflectate specular pentru fragmentele interioare unei fațete

(intensitatile fragmentelor interioare nu pot fi mai mari decat cele din varfuri)

Modelul Phong

Dezavantaje fata de modelul Gouraud

1. Componentele Nx, Ny, Nz ale normalei unui fragment se obţin printr-un calcul incremental (analog cu cel folosit pentru calculul culo rilor in modelul Gouraud) dar, pentru folosirea lor în calculul culorii, normala trebuie să fie normalizată:

Nu = N/|N|,

unde **|N| = (Nx 2 + Ny 2 + Nz 2)^0.5**

2. De asemenea, vectorii L (catre sursa de lumina) si V (catre observator), folositi in modelul de iluminare locala la nivel de fragment, sunt calculati si normalizati in programul fragment shader pentru a fi folositi in calculul culorii fragmentului.

3. Calcule mai complexe la nivel de fragment decat in modelul Gouraud

Avantaje

1. Permite redarea reflexiei speculare in orice fragment al unui poligon
2. Reduce mult efectul de banda Mach

14. Comparatie intre modelul Gourraud si modelul Phong

Am scris mai sus.(13)

15. Calculul culorilor fragmentelor la afisarea suprafetelor 3D folosind modelul Phong. Avantajele si dezavantajele modelului Phong in comparatie cu alte modele.

Calculul culorii fragmentelor

La momentul rasterizarii, fiecarui vârf al unui poligon îi este asociata normala în vârf , care poate fi calculata la fel ca pentru modelul Gouraud. Normalele sunt calculate si asociate
varfurilor în programul de aplicatie.

• Pentru fiecare fragment rezultat din rasterizarea unui poligon, GPU calculează o normala prin interpolare liniara intre normalele varfurilor.

• Normala interpolata este intrare pentru programul fragment shader.

• In fragment shader se calculeaza culoarea fragmentului pe baza normalei interpolate, folosind modelul de iluminare locala.

La rasterizarea unui poligon se calculeaza o normala pentru fiecare fragment astfel:

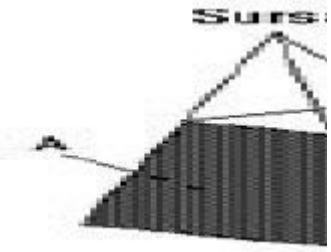
- prin interpolarea liniara a normalelor varfurilor, pentru fragmentele de pe laturi (la fel ca in modelul Gouraud, pentru culori);

- prin interpolare liniara intre normalele capetelor fiecarui segment interior, pentru fragmentele interioare poligonului (la fel ca in modelul Gouraud, pentru culori).

Avantaje/Dezavantaje ->14

16. Introducerea umbrelor in imagini prin metoda volumelor de umbra (fara implementare cu OpenGL).

- sursa de lumina considerata punctiforma, scena 3D alcatuita din poligoane
- volum de umbra: definit de o sursa de lumina si un poligon luminat (vizibil din pozitia sursei)
- **poligon generator** = poligonul luminat



- volumul infinit este determinat de o sursa si un poligon generator; este delimitat de o fata aflata la o anumita distanta fata de sursa, distanta dincolo de care intensitatea luminii este neglijabila. Acea fata reprezinta poligonul generator, t
- volumul de umbra este decupat la marginile volumului vizual
- **poligoane de umbra** = fete laterale ale volumului; normalele la suprafete - indreptate spre exterior

Fie:

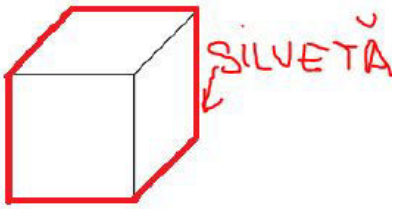
- PUV - poligoane vizibile observatorului (A si B in figura)
- PUN - nevizibile obs (C in desen)

Clasificarea poligoanelor de umbra in PUV si PUN de poate face pe baza normalelor (back face culling).

Pe foarte scurt: fie P un punct al unei suprafete si VP vectorul din pozitia

observatorului in P. **Daca V nu e in umbra**, P este umbrit ⇔ nr. PUV intersectate de VP > nr. PUN intersectate.

Optimizare: in loc sa calculam volumul de umbra pt fiecare poligon generator, vom calcula un singur volum de umbra pt o suprafata poliedrala => se vor determina poligoane de umbra doar pt laturile care fac parte din silueta unei suprafete (si anume doar pt cele luminate).
latura de silueta = margine a unei suprafete sau latura care separa un poligon vizibil de unul nevizibil



Se implementeaza folosind stencil buffer de pe GPU, impreuna cu z-buffer. Pixelii aflati in umbra au valori nenule in buffer.
Pixelii de umbra au culoarea:
Cfragment = Cemisiva + Cambianta = ke +ka * globalAmbient

Aprecieri asupra metodei volumelor de umbra

Pozitive:

- Este generala si poate fi aplicata pentru orice scena, pentru oricate surse de lumina.
- Produce forme de umbre exacte.

Negative:

- Complexa din punct de vedere al timpului de calcul:
  se rasterizeaza scena de 2 ori si poligoanele de umbra de 2 ori!
- Sursele de lumina sunt considerate punctiforme, de aceea nu pot fi obtinute penumbre.

17. Introducerea cetei in imagini :

1. Cum se modifica culoarea unui fragment tinand cont de factorul de ceata si culoarea de ceata?

Fără ceaţă, pe măsură ce observatorul se îndepărtează de un obiect, obiectul se aproprie de planul din spate şi deci este decupat atunci când planul din spate îl intersectează.

Cu ceaţă, dacă intensitatea ceţii creşte proporţional cu distanţa de la poziţia

observatorului, se crează efectul de distanţă pentru obiectele îndepărtate.

Dacă densitatea ceţii creşte până la opacitate la o distanta mai mica decat aceea a planului din spate, decuparea faţă de planul din spate nu mai este necesară căci obiectele dispar datorită opacităţii.

Cfog–culoarea de ceaţă

Cfragment–culoarea calculată pentru un fragment

0<= Φ <= 1 –factorul de ceaţă, proporţional cu distanţa de la observator la punctul din spaţiul 3D vizibil in pixel (Φ=1: opacitate)

Atunci, Cpixel, culoarea de afişare a pixelului corespunzator fragmentului este:

Cpixel= (1-Φ)*Cfragment+ Φ*Cfog

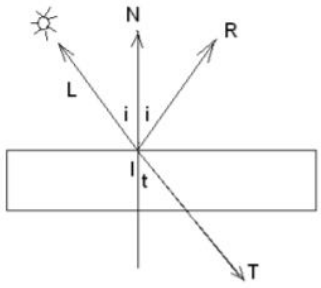
2. Cum se calculeaza factorul de ceata?

Factorul de ceaţă liniar (obtinut prin interpolare liniara) actioneaza intre 2 distante definite in sistemul coordonatelor observator, zo si z1, masurate pe directia de observare.
Factorul de ceata este calculat in timpul rasterizarii, pentru fiecare fragment, folosind coordonata z a fragmentului si distantele z0 si z1 transformate in coordonate ale spatiului de afisare:
z: coordonata z a fragmentului
Φ - Factorul de ceata
Φ = 0 pentru z < z0
Φ = (z-z0) / (z1-z0) pentru z0 <= z <= z1
Φ = 1 pentru z > z1
Valorile implicite pentru z0 si z1 sunt 0 si 1, in spatiul de afisare: coordonatele z ale planului din fata si planului din spate al vederii, dupa transformarea in poarta de afisare.

Factorul de ceata moduleaza culoarea fragmentului dupa texturare.

18. Ray Tracing:

- **Care sunt razele folosite in calculul culorii unui pixel in algoritmul Ray Tracing?**



L: raza din I catre sursa de lumina

R: raza reflectata specular, simetrica, fata de N, cu L

T: raza transmisa, calculata pe baza legii lui Snell:

- **Cum se calculeaza directia lor?**

$R = 2(N \cdot L) \cdot N - L$

$n1/n2 = \sin(t)/\sin(i)$, n1,n2: indicii de refractie

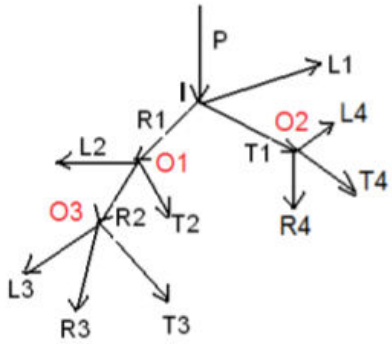
$T = L*(n1/n2) - (\cos(t) + (n1/n2)*(L \cdot N))*N$

- **Formula de calcul a culorii unui pixel?**

Calculul culorii luminii în punctul I:

$$I_\lambda(I) = I_{local\lambda}(I) + Ks*R_\lambda(I) + Kt*T_\lambda(I)$$

- **Care este arborele de raze?**

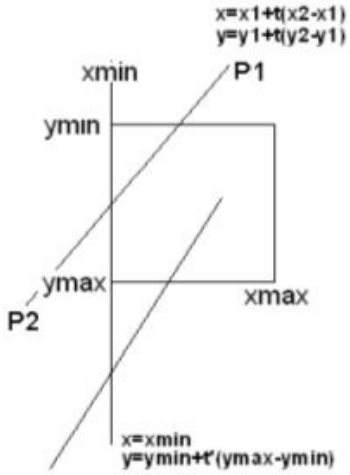


Arborele de raze

➢*Pentru obtinerea culorii pixelului, se evalueaza arborele de raze de la frunze catre radacina*

19. Pentru decuparea vectorilor 2D: Descriere, Pasii algoritmului pentru decuparea vectorului din figura (vezi sus figura cu un patrat taiat de o linie)

Calculul intersectiilor cu laturile dreptunghiului de decupare



Intersectia segmentului P1-P2 cu latura x = xmin a dreptunghiului

Ec. parametrica a vectorului P1-P2:

P(t) = P1 + (P2-P1)*t 0<= t <=1 intre P1-P2

Punctul de intersectie cu dreapta x=xmin are abscisa xmin:

1) xmin=x1+t(x2-x1) → t=(xmin-x1)/(x2-x1)

2) este 0<=t<=1 ???

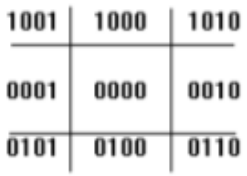
da: punctul de intersectie se afla pe segmentul P1-P2

- se calculeaza yi=y1+(xmin-x1)*(y2-y1)/(x2-x1)
- este ymin<= yi <= ymax ??

da: exista intersectie intre vectorul P1-P2 si latura x=xmin

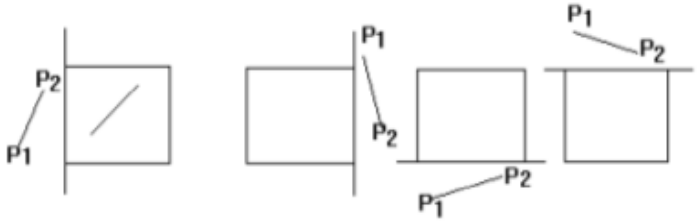
nu: punct de intersectie cu prelungirea laturii – calcul inutil

20. Decuparea vectorilor in algoritmul Cohen-Sutherland (fara implementare)



- Punctele din plan sunt codificate prin 4 biti, in functie de pozitia lor fata de dreptunghiul de decupare. De exemplu: b3b2b1b0
- b0=1: pentru punctele din stanga dreptunghiului,
- b1=1: dreapta, b2=1: sub dreptunghi, b3=1: deasupra dreptunghiului

Fie **cod1** codul binar al punctului P1 si **cod2** codul binar al punctului P2.

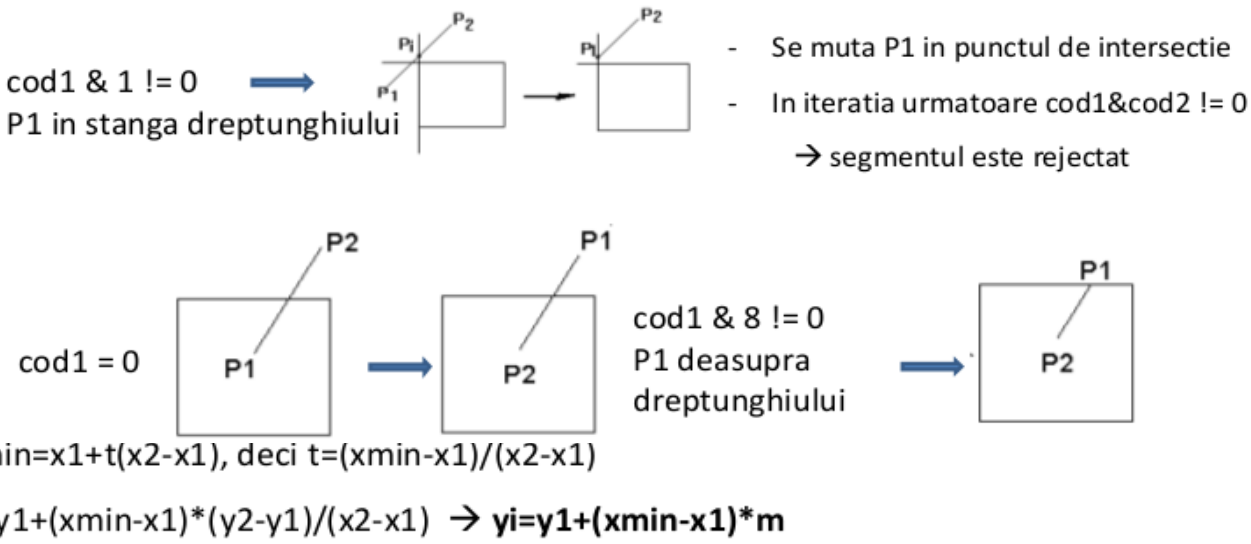


1) cod1 ==0 && cod2==0 :
segment acceptat trivial

2) (cod1 & cod2) != 0: segment rejectat trivial (cele 2 coduri au acelasi bit =1)
segmentul nu intersecteaza dreptunghiul

3) altfel: se intersecteaza segmentul P1-P2 cu dreptunghiul de decupare

- Alegerea laturii cu care se intersecteaza segmentul P1-P2: pe baza codului varfului P1, care trebuie sa fie situat in afara dreptunghiului de decupare.
- Daca cod1 = 0 → se interschimba P1 cu P2



xmin=x1+t(x2-x1), deci t=(xmin-x1)/(x2-x1)

y=y1+(xmin-x1)*(y2-y1)/(x2-x1) → **yi=y1+(xmin-x1)*m**

21. Algoritmul Bresenham pentru rasterizarea vectorilor: alegerea punctelor spatiului discret, implementare in C, generalizarea algoritmului pentru vectori de orice panta.

-
- *Alegerea punctelor spatiului discret.*
  - O si D sunt adrese de pixeli (puncte ale spatiului discret)
  - Următorul punct al spatiului discret ales pentru aproximarea vectorului va fi:
    - O dacă d1<d2, D în caz contrar.
    - Dacă d1=d2 se poate alege oricare dintre cele două puncte

- *Deducerea valorii expresiei de test.*
  - Valoarea variabilei de test se obtine prin calcul incremental: adunarea unei constante intregi

Implementare in C

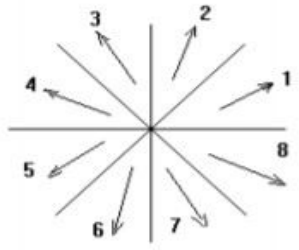
```
void Bres_vect(int x1, int y1, int x2, int y2)
{ //pentru vectori cu panta cuprinsă între 0 şi 1
 int dx, c1, c2, x, y, t;
 dx=x2-x1;
 c1=(y2-y1)<<1;  // 2*dy
 c2=c1-(dx<<1);  // 2*dy – 2*dx
 t=c1-dx;  // 2*dy - dx
 putpixel(x1, y1);
 for(x=x1+1, y=y1; x<x2; x++)
  { if(t<0) t+= c1; //deplasament  O
     else  { t+= c2; y++;} // deplasament D
   putpixel(x,y);
  }
 putpixel(x2,y2);
}
```

- ***Generalizarea algoritmului pentru vectori de orice panta.***

Generalizarea algoritmului Bresenham pentru vectori de orice panta

▪ Vectorii definiţi în planul XOY pot fi clasificaţi, pe baza pantei, în opt clase geometrice, numite **"octanţi"**

▪ Un vector care apartine unui octant O are 7 vectori simetrici in ceilalti 7 octanti



dx=x2 – x1 şi dy=y2 – y1

octantul 1: dx>0 şi dy>0 şi dx>=dy;

octantul 2: dx>0 şi dy>0 şi dx<dy;

octantul 3: dx<0 şi dy>0 şi abs(dx)<dy;

octantul 4: dx<0 şi dy>0 şi abs(dx)>=dy;

octantul 5: dx<0 şi dy<0 şi abs(dx)>=abs(dy);

octantul 6: dx<0 şi dy<0 şi abs(dx)<abs(dy);

octantul 7: dx>0 şi dy<0 şi dx<abs(dy);

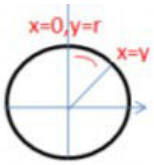
octantul 8: dx>0 şi dy<0 şi dx>=abs(dy);

22. Algoritmul Bresenham pentru rasterizarea cercurilor:

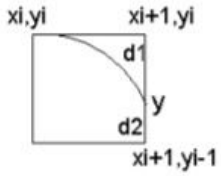
alegerea punctelor spatiului discret, implementare in C, functia de afisare a punctelor de pe cerc.

- *alegerea punctelor spatiului discret:*

- Consideram cercul cu centrul în originea sistemului de coordonate si raza r.



- In cadrul algoritmului **se calculeaza numai punctele din octantul al 2-lea,** celelalte obtinandu-se prin simetrie.
- Se calculeaza punctele de pe cerc incepand cu (x=0,y=r) pana la (x=y), prin incrementarea lui x.
- Fie (x_i,y_i) ultimul punct al spatiului discret, ales pentru aproximarea cercului. Urmatorul punct va fi unul dintre (x_i+1,y_i) si (x_i+1,y_i-1):



Fie y ordonata punctului de pe cercul teoretic, cu abscisa x_i+1.

$$y^2 = r^2-(x_i+1)^2$$

Fie d1 si d2 distantele de la cele 2 puncte ale spatiului discret la punctul de pe cercul teoretic.

Intereseaza semnul diferentei (d1-d2):

- daca $(y_i - y) > (y- (y_i-1))$ atunci

$$(y_i^2 - y^2) > (y^2 - (y_i-1)^2)$$

$$d1' = y_i^2 - y^2 = y_i^2 - r^2 + (x_i+1)^2$$

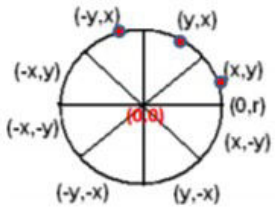
$$d2' = y^2 - (y_i-1)^2 = r^2 - (x_i+1)^2 - (y_i-1)^2$$

- *implementare in C*

```
void Bres_cerc(int xc, int yc, int r, RGB culoare)
{ int x,y,t;
   t=3-(r<<1);
   point(xc+r,yc,culoare); point(xc-r,yc,culoare);
   point(xc,yc+r,culoare); point(xc,yc-r,culoare);
   for(x=1,y=r; x<y; x++)
   {
     if(t<0)
       t+=6+(x<<2);
     else
      { t+=10+((x-y)<<2); y--;}
     punct_simetric(xc,yc,x,y,culoare);
   }
// pentru x==y se afiseaza 4 puncte simetrice
  point(xc+x,yc+y,culoare); point(xc+x,yc-y,culoare);
  point(xc-x,yc+y,culoare); point(xc-x,yc-y,culoare);
}
```

- *functia de afisare a punctelor de pe cerc. (e singura chestie care parea mai aproape de intrebare - slide 6)*

```
// Afiseaza punctele simetrice de pe cerc
void punct_simetric(int xc,int yc,int x,int y, RGB culoare)
{ point(xc+x,yc+y,culoare);
  point(xc+x,yc-y,culoare);
  point(xc-x,yc-y,culoare);
  point(xc-x,yc+y,culoare);
  point(xc+y,yc+x,culoare);
  point(xc+y,yc-x,culoare);
  point(xc-y,yc-x,culoare);
  point(xc-y,yc+x,culoare);
}
```



23. Transformarea de vizualizare in OpenGL. Parametrii care definesc sistemul de coordonate observatoare. Care este functia de OpenGL care permite definirea acestor parametrii?

Din sistemul de coordonate globale --> in sistemul de coordonate observator

Definirea s istemului de coordonate observator (sistemul atasat camerei virtuale)

Sistem de coordonate 3D dreapta, definit prin 3 parametri:

- Originea sa: pozitia camerei virtuale in spatiul coordonatelor globale (xo, yo, xo).

- Directia in care priveste observatorul (spre scena 3D) – vectorul Look, reprezinta normala la planul de vizualizare (N).

- Directia “sus” a planului de vizualizare (vectorul Up), care determina directia axei OY a sist. de coordonate observator: reprezinta rotatia camerei in jurul axei privirii (Look)

Transformarea de vizualizare in OpenGL:

Pentru setarea zonei de afisare in OpenGL se apeleaza

void glViewport(GLint px, GLint Py, GLsizei width, GLsizei height);

(px, py) reprezintă coordonate relative la coltul stanga-jos al fereastrei aplicatiei (în pixeli). Valorile implicite : (0,0).

width, height reprezintă lăţimea/ înălţimea porţii de afişare.

Valorile implicite sunt: lăţimea şi înălţimea ferestrei aplicatiei.

Orientarea axelor viewportului este aceeasi cu a sistemului coordonatelor logice.

24. Forma generala (in pseudocod) a algoritmului de rasterizare a suprafetelor poligonale. Care e definitia punctului interior a unui poligon, pe care se bazeaza algoritmul? Cate puncte de intersectie trebuie sa se obtina atunci cand o linie raster traverseaza un varf al poligonului?

Un punct este interior daca orice segment dus din el intersecteaza un numar impar de laturi. Pentru cele de maxim local nu se interseccteaza nicio latura, iar pentru cele de minim local se intersecteaza 2.

```
typedef struct{int x,y;}virf;


struct latura{ int ymax, ymin;
        float xi,dx;
        struct latura * urm;
};
typedef struct latura latura;
latura * prim,*ult;
latura **ord;



Initial: ult=prim;
int set_activ( int yr)
{
* Adaugă laturile care devin active
        ( avans ult in lista laturilor cat timp yr ==ymin_latura)
* Elimină din lista laturilor, intre prim si ult, laturile care au devenit inactive:
        (yr == ymax_latura)
        return 1 daca s-a modificat setul laturilor active,
        0 altfel;
}
```

```
int colorare_poligon(int nrv, virf poly[])
{ // nrv este numarul de vârfuri ale poligonului; poly este vectorul vârfurilor
        latura **ord; latura * k; int I, k, modif, x;
* Creaza lista de laturi ordonata crescator dupa ymin_laturi;
        ult = prim;

        for(yr=ymin_poligon; yr < ymax_poligon; yr++) {
                modif=set_activ(yr); // calculeaza setul laturilor active
* Calculeaza punctele de intersecţie cu laturile active (memorate in celulele listei de laturi
intre prim si ult): xi = xi + dx
        if(modif) // s-a modificat setul laturilor active
* Memoreaza in vectorul ord pointeri catre laturile active

// Ordoneza vectorul ord a.i. sa puncteze catre punctele de intersecţie in ordinea crescatoare a
absciselor
do {o=0;
        for(i=0; i<nrint-1; i++) // nrint este numarul de puncte de intersectie
        if(ord[i]->xi > ord[i+1]->xi) {k=ord[i]; ord[i]=ord[i+1]; ord[i+1]=k;o=1;}
}while(o);
//Coloreaza pixelii interiori poligonului de pe linia curenta
        for(i=0; i<nrint-1; i+=2)
                for(x = (int)(ord[i]->xi+0.5); x<=(int)(ord[i+1]->xi+0.5); x++)
                {setpixel(x, yr, culoare(x,yr)); // afiseaza fragmentul
}//for yr
return 1;
```

(Examen 6 Februarie 2017)

25. Modelul de reflexie Phong(modelul de iluminare locala) pentru calculul luminii reflectate intr-un punct al unei suprafete 3D. Precizati care sunt componentele luminii reflectate si care sunt calculate.

Componenta ambientală

Aceasta reprezintă lumina reflectată de catre obiectele din scenă de atât de multe ori încât pare să vină de peste tot.

```
ambientala = Ka * culoareaAmbientalaGlobala
```

Componenta difuză

Aceasta reprezintă lumina reflectată de suprafața obiectului în mod egal în toate direcțiile.

```
difuză = Kd * culoareLumina * max (N * L, 0)
```

d - constanta de reflexie difuza a materialului

•culoareLumina – culoarea difuză a luminii

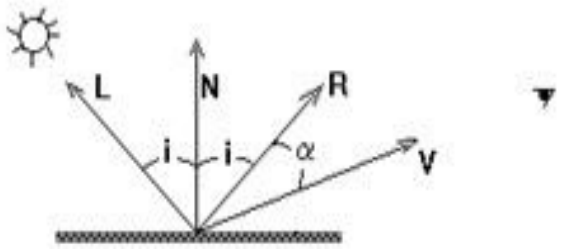
•N – normala la suprafață (normalizată)

•L – vectorul direcției luminii incidente (normalizat)

•max(N·L,0)– produsul scalar N·L reprezintă măsura unghiului dintre acești 2 vectori

Componenta speculara

Un reflector perfect, de exemplu o oglindă, reflectă lumina numai într-o singură direcție R, care este simetrică cu L față de normala la suprafață. Prin urmare, doar un observator situat exact pe direcția respectivă va percepe raza reflectată.



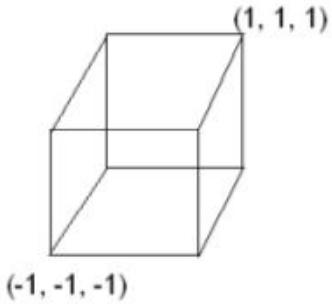
Componenta specular reprezintă lumina reflectată de suprafața obiectului numai în jurul acestei

direcţii,R.

26. Care este matricea prin care se transforma un varf din sistemul coordonator obiect in sistemul coordonator de decupare? Care este volumul de decupare? Cand este efectuata decuparea primitivelor in banda grafica?

Matricea MVP(executand Vertex Shader in paralel)

Volumul de decupare



Faţă de volumul vizual canonic, un punct din spatiu se poate afla: in interior, in stanga, in dreapta, sub volum, deasupra volumului, in fata, in spate:

→pentru codificarea pozitiei unui punct din spatiu fata de volumul vizual canonic sunt necesari 6 biti.

De ex. se poate face conventia:

b0 = 1 pentru puncte (x,y,z) cu x < -1

b1 = 1 pentru puncte (x,y,z) cu x > 1

b2 = 1 pentru puncte (x,y,z) cu y < -1

b3 = 1 pentru puncte (x,y,z) cu y > 1

b4 = 1 pentru puncte (x,y,z) cu z < -1

b5 = 1 pentru puncte (x,y,z) cu z > 1

daca: cod(P1) ==0 && cod(P2)==0
    segment acceptat trivial

daca (cod(P1) & cod(P2)) != 0
    segment rejectat trivial
altfel
    se intersecteaza segmentul cu volumul

Decuparea: înainte sau dupa împarţirea perspectivă?

1. Inainte: decupare in coordonate omogene
2. Dupa: decupare in coordonate carteziene (NDC)

27. Care sunt parametrii prin care este definit sistemul de coordonate observator(atasat camerei virtuale)? Care sunt

valorile implicite ale acestor parametrii? Care este produsul de matrici prin care se obtine matricea transformarii de vizualizare(din sistemul de coordonate globale in sistemul de coordonate observator).

Definirea s istemului de coordonate observator (sistemul atasat camerei virtuale)

❖ Sistem de coordonate 3D dreapta, definit prin 3 parametri:

- Originea sa: pozitia camerei virtuale in spatiul coordonatelor globale (xo, yo, xo).
- Directia in care priveste observatorul (spre scena 3D) – vectorul Look, reprezinta normala la planul de vizualizare (N).

-Directia “sus” a planului de vizualizare (vectorul Up), care determina directia axei OY a sistemului de coordonate observator: reprezinta rotatia camerei in jurul axei privirii (Look)

Sistemul de coordonate observator implicit in OpenGL

▪ poziţia observatorului (camerei): originea sistemului de coordonate globale

▪ direcţia în care priveşte observatorul: direcţia negativa a axei OZ

▪ direcţia sus a planului de vizualizare: direcţia pozitivã a axei OY

Cu aceste valori implicite, transformarea de vizualizare estetransformarea identica.

Matricea transformarii de vizualizare: V = Rz(u) * A(k,kr) * T(-xo,-yo,-zo)

-A(k,kr) este o transformare prin care se aliniaza axele z ale celor 2 sisteme de coordonate: consta din 2 rotatii – o rotatie in jurul axei X si una in jurul axei Y.

-Prin rotatia in jurul axei z (Rz) se aliniaza celelalte 2 axe.

28. Care sunt caracteristicile proiectiilor axonometrice? Scrieti produsul de matrici prin care se poate obtine proiectia axonometrica a unui punct.

Caracteristici → 1, 2

2 posibilitati de exprimare matematica:

- Proiectie ortogonala intr-un plan oarecare – obiect fix in spatiu, alegere plan astfel incat sa obtinem cele 3 cazuri (izometrice, dimetrice, generale):

[xp yp zp 1] = [x y z 1]* Pax(N,R0)

- Transformare geometrica a obiectului (TA) urmata de proiectie ortografica in XOY (Po-xoy)– cu efect echivalent – plan de proiectie fix (XOY), pozitionare obiect in spatiu:

[xp yp zp 1] = [x y z 1]* TA * Po-xoy = [x y z 1]* PA

TA consta din rotatii in jurul axelor OX si OY. Particularizand unghiurile de rotatie obtinem cazurile de proiectii axonometrice: izometrice, dimetrice, generale.

Proiectii axonometrice(2)

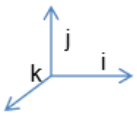
Exemplu:

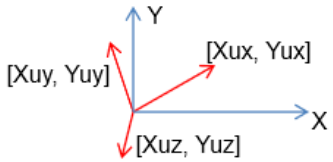
TA = Ry(uy) * Rx(ux)

$$PA = TA*Po\text{-}xoy = \begin{bmatrix} \cos(uy) & \sin(ux)*\sin(uy) & 0 & 0 \\ 0 & \cos(ux) & 0 & 0 \\ \sin(uy) & -\cos(uy)*\sin(ux) & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Se aplica transformarea PA versorilor axelor principale:

$$U * PA = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} * PA = \begin{bmatrix} Xux & Yux & 0 & 1 \\ Xuy & Yuy & 0 & 1 \\ Xuz & Yuz & 0 & 1 \end{bmatrix}$$





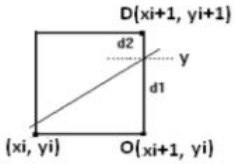
Factorii de scalare a laturilor pe cele 3 axe sunt lungimile proiectiilor celor trei vectori unitate:

$$sx = (Xux^2 + Yux^2)^{0.5}$$
$$sy = (Xuy^2 + Yuy^2)^{0.5}$$
$$sz = (Xuz^2 + Yuz^2)^{0.5}$$

29. Cum se aleg pixelii de pe traseul unui vector in algoritmul Bresenham pentru rasterizarea vectorilor? Cum se generalizeaza algoritmul pentru rasterizarea vectorilor de orice panta?

Algoritmul Bresenham (1)

- Contine numai operatii cu numere intregi. **Este definit pentru vectori din primul octant**
- Un vector face parte din"primul octant" daca panta sa, m, satisface conditia 0<m<1.
- Coordonatele pixelilor de pe traseul vectorului se obtin prin calcul incremental.
- Pentru fiecare valoare a lui x, de la xmin_vector la xmax_vector, se alege acel punct al spaţiului discret care este mai apropiat de punctul de pe vectorul teoretic.



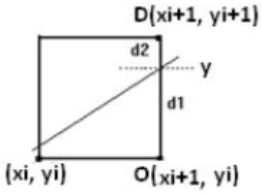
- Fie m=(y2-y1)/(x2-x1) panta vectorului,
- (x_i, y_i) ultimul punct al spaţiului discret ales la executia algoritmului
- d1 distanţa de la punctul de pe vectorul teoretic, (x_i+1, y), la punctul $O(x_i+1,y_i)$
- d2 distanţa de la punctul de pe vectorul teoretic la punctul $D(x_i+1,y_i+1)$.
- O si D sunt adrese de pixeli (puncte ale spatiului discret)

➢ Următorul punct al spatiului discret ales pentru aproximarea vectorului va fi:

- O dacă d1<d2, D în caz contrar.
- Dacă d1=d2 se poate alege oricare dintre cele două puncte.

- Exprimăm diferetja d1-d2:

  $y=m*(x_i+1)+b$ este ordonata punctului de pe vectorul teoretic



  $d1= y - y_i= m*(x_i+1)+b-y_i$

  $d2= y_i + 1-y= y_i+1-m*(x_i+1)-b$

  $d1- d2= 2*m*(x_i+1)-2*y_i+2*b -1$

- Se înlocuieşte m cu dy/dx apoi se înmulţeşte în ambele părţi cu dx. Rezultă:

$$t_i= (d1-d2)*dx= 2*dy*(x_i+1)-2*dx*y_i+2*b*dx-dx = 2*dy*x_i - 2*dx*y_i + 2*b*dx - dx + 2*dy$$

- t_i reprezintă eroarea de aproximare în pasul i: pe baza ei se alege urmatorul punct al spatiului discret

  Notăm cu (x_{i+1},y_{i+1}) punctul care se va alege în pasul i.

- Expresia erorii de aproximare în pasul i+1 este:

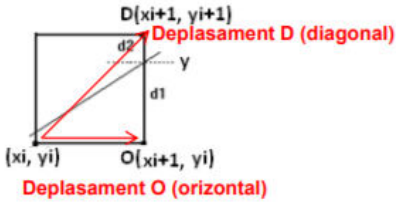
$$t_{i+1} = 2*dy*x_{i+1} - 2*dx*y_{i+1} + 2*b*dx - dx +2*dy$$

contribuie la culoarea unui pixel si cum sunt evaluate)

3 Afisarea in umbra a partilor unui obiect.

1 Sol: Ex 7 (prima parte)

$t_i=(d1-d2)*dx = 2*dy*x_i - 2*dx*y_i + 2*b*dx-dx + 2*dy$, (dx>0)



(1) Dacă $t_i <= 0$, se alege punctul O, deci $x_{i+1} = x_i+1$ şi $y_{i+1} = y_i$

Rezultă:

$t_{i+1} = 2*dy*(x_i+1) - 2*dx*y_i + 2*b*dx - dx + 2*dy$

sau **$t_{i+1} = t_i + 2*dy$**

(2) Dacă $t_i > 0$, se alege punctul D, deci $x_{i+1} = x_i + 1$ şi $y_{i+1} = y_i + 1$

Rezultă:

$t_{i+1} = 2*dy*(x_i+1) - 2*dx*(y_i+1) + 2*b*dx - dx + 2*dy$

sau **$t_{i+1} = t_i + 2*dy - 2*dx$**

- Valoarea variabilei de test se obtine prin calcul incremental: adunarea unei constante intregi

Eroarea de aproximare pentru primul pas: $x_i = x1$ şi $y_i = y1$:

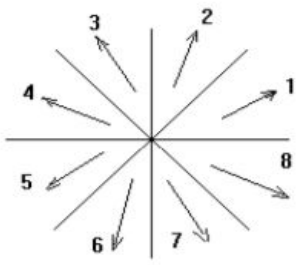
$t_1 = 2*dy*x1 - 2*dx*y1 + 2*dy - dx + 2*dx(y1 - (dy/dx)*x1)$

sau **$t_1 = 2*dy-dx$**

Algoritmul Bresenham- generalizare(1)

Generalizarea algoritmului Bresenham pentru vectori de orice panta

- Vectorii definiţi în planul XOY pot fi clasificaţi, pe baza pantei, în opt clase geometrice, numite **"octanţi"**
- Un vector care apartine unui octant O are 7 vectori simetrici in ceilalti 7 octanti



dx=x2 – x1 şi dy=y2 – y1

octantul 1: dx>0 şi dy>0 şi dx>=dy;

octantul 2: dx>0 şi dy>0 şi dx<dy;

octantul 3: dx<0 şi dy>0 şi abs(dx)<dy;

octantul 4: dx<0 şi dy>0 şi abs(dx)>=dy;

octantul 5: dx<0 şi dy<0 şi abs(dx)>=abs(dy);

octantul 6: dx<0 şi dy<0 şi abs(dx)<abs(dy);

octantul 7: dx>0 şi dy<0 şi dx<abs(dy);

octantul 8: dx>0 şi dy<0 şi dx>=abs(dy);

Notam cu:

+h, deplasamentul orizontal spre dreapta (în sensul crescător al axei x),

-h, deplasamentul orizontal spre stânga (în sensul descrescător al axei x),

+v, deplasamentul vertical în sus (în sensul crescător al axei y),

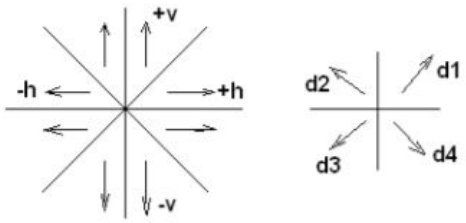
-v, deplasamentul vertical în jos (în sensul descrescător al axei y),

d1, deplasamentul diagonal dreapta-sus,

d2, deplasamentul diagonal stânga-sus,

d3, deplasamentul diagonal stânga-jos,

d4, deplasamentul diagonal dreapta-jos.



Octant	1	2	3	4	5	6	7	8
Deplas O	+h	+v	+v	-h	-h	-v	-v	+h
Deplas D	d1	d1	d2	d2	d3	d3	d4	d4

Corespondenţa între alegerea curentă într-un pas al algoritmului Bresenham (punctul O sau punctul D) şi deplasamentul echivalent în fiecare octant:

30. Cum sunt efectuate urmatoarele operatii in algoritmul Ray-tracing:

1. **Eliminarea partilor nevizibile.**
2. **Calculul culorii pixelilor intr-un cadru imagine(precizati care sunt componentele luminii care contribuie la culoarea unui pixel si cum sunt evaluate)**
3. **Afisarea in umbra a partilor unui obiect.**

Sol: Ex 7 (prima parte)

Imaginea se calculeaza pixel cu pixel, pornind de la spatiul imagine.

Considerand o singura sursa de lumina:

Pentru fiecare pixel al imaginii:

- Se calculeaza raza primara, P, care porneste din pozitia observatorului si trece prin centrul pixelului, in planul imaginii
- Se determina punctul de intersectie, I, al razei primare cu cel mai apropiat obiect de observator
- **Daca** raza nu intersecteaza nici un obiect al scenei,
  - Se afiseaza pixelul in culoarea fondului

  altfel
  - Se calculeaza culoarea obiectului in punctul I, tinand cont de:
    - Reflexia luminii provenita direct de la sursa de lumina, raza L
    - Lumina provenita in I de la alte obiecte pe directia reflexiei speculare, raza R
    - Lumina provenita in I de la alte obiecte pe directia razei transmise, raza T
  - Afiseaza pixelul in culoarea obtinuta prin combinarea contributiei celor trei raze

3.

Atunci când un observator priveşte o scenă 3D luminată de o sursă de lumină dintr-o poziţie diferită de aceea a sursei de lumină, va vedea umbrele produse de obiectele scenei.

Umbrele au o contribuţie însemnată la realismul imaginii, îmbunătăţind percepţia profunzimii.

In general, se considera numai umbra pura: calcule mai simple.

Volumul de calcule depinde şi de poziţia sursei de lumină:

▪ Dacă sursa este la infinit calculele sunt mai simple.

▪ In cazul unei surse situate la distanţă finită dar în afara volumului vizual, este necesară o proiecţie perspectivă din poziţia sursei.

▪ Cazul cel mai dificil este acela în care sursa este situată în volumul vizual.

Problema determinării umbrelor este similară aceleia de determinare a vizibilităţii obiectelor: partile scenei 3D care nu sunt vizibile din poziţia sursei de lumină sunt în umbră.

◊ Crearea unei imagini cu umbre presupune rezolvarea de două ori a problemei suprafeţelor nevizibile: o dată privind scena din poziţia sursei de lumină, a doua oară privind-o din poziţia observatorului.

Sunt două tipuri de umbre: umbre proprii şi umbre purtate.

–Umbrele proprii sunt generate de obiectul însuşi, care împiedică lumina să ajungă la unele dintre feţele sale: feţele umbrite de umbra proprie sunt feţele auto-obturate atunci când scena este văzută din poziţia sursei de lumină.

–Umbra purtată este umbra pe care un obiect o produce pe alte părţi ale scenei, la care lumina nu ajunge din cauza obiectului.

31. Cum poate fi utilizat algoritmul BSP pentru eliminarea obiectelor nevizibile din banda grafica (Object culling)?

Initial, arborele BSP a fost folosit numai pentru operatia de eliminare a obiectelor (grupurilor de primitive) nevizibile din banda grafica (object culling).

❖ Ulterior a devenit algoritm pentru reprezentarea si afisarea scenelor 3D cu "eliminarea" partilor nevizibile. ➢ Primitivele grafice sunt trimise in banda grafica in ordinea "din spate in faţă": nu mai este necesar testul de adâncime la rasterizarea primitivelor grafice (testul de adâncime poate fi dezactivat: fragmentele care se proiecteaza in acelasi pixel sunt suprascrise).

❖ Este eficient pentru eliminarea partilor nevizibile ale poligoanelor (hidden surface removal) dar poate fi folosit si pentru eliminarea partilor nevizibile ale liniilor dintr-o scena compusa din poligoane si linii.

Intrarea: lista primitivelor care compun scena 3D (nu are importanta din care obiect face parte fiecare primitiva)

–Scena 3D este reprezentata printr-un arbore binar: arborele BSP

–Arborele BSP al scenei este independent de pozitia observatorului (reprezinta scena in sistemul coordonatelor globale).

Construirea arborelui BSP al unei scene

▪Fiecare nod al arborelui corespunde unui plan de partitionare a spatiului 3D (de regula, planul unui poligon al scenei)

▪ Fiecare plan de partitionare imparte spatiul in 2 semi-spatii: –cel din faţa

planului (de aceeasi parte cu normala la plan) –cel din spatele planului ▪ Se incepe cu un poligon oarecare din lista (de regula primul),

pentru care se creaza nodul radacina al arborelui

▪ Primitivele din semi-spatiul “faţă” formeaza “lista-faţă”,

care va genera subarborele “faţă” al nodului

▪ Primitivele din semi-spatiul “spate” formeaza “lista-spate”,

care va genera subarborele “spate” al nodului.

▪ Se alege un poligon din lista-faţă si se creaza nodul radacina al subarborelui “faţă, etc.